Планировщик отпусков

КОРПОРАТИВНЫЙ ГРАФИК ОТПУСКОВ И СОГЛАСОВАНИЙ

Регламент резервного копирования

Копии, восстановление и ежемесячная проверка · версия 1.0.0

30 октября 2026

© Alexander Yurasov

Содержание

1 О документе 3

2 Когда применять 4

3 Что копируется 5

4 Перед началом 6

5 Проверка свежести копий 7

5.1 1. Логические копии 7

5.2 2. Копия вне сервера 7

5.3 3. Физические копии, архив WAL и реплика 7

6 Внеочередная копия 8

6.1 1. Снимите копию 8

6.2 2. Проверьте, что копия читается 8

6.3 3. Вынесите копию с сервера 8

7 Полное восстановление 9

7.1 1. Подготовьте копию и список потерянных изменений 9

7.2 2. Остановите приложение и копирование 9

7.3 3. Пересоздайте базу и восстановите копию 9

7.4 4. Запустите приложение 9

7.5 5. Проверьте 9

7.6 Восстановление на новом узле 10

8 Восстановление на момент времени 11

8.1 1. Определите момент T 11

8.2 2. Найдите том архива и физическую копию 11

8.3 3. Разверните копию в пустой каталог 11

8.4 4. Запустите восстановление 11

8.5 5. Сверьте данные и завершите восстановление 11

8.6 6. Перенесите результат 12

8.7 7. Уберите временный экземпляр 12

9 Ежемесячная проверка восстановления 13

9.1 1. Восстановите последнюю копию 13

9.2 2. Сверьте с рабочей базой 13

9.3 3. Удалите проверочную базу 13

9.4 4. Заполните протокол 13

10 Ежеквартальная проверка восстановления на момент времени 14

11 Проверка 15

12 Если не получилось 16

13 Запись в журнал эксплуатации 17

1 О документе

Регламент резервного копирования и восстановления «Планировщика отпусков» 1.0.0 (E52-04): что и где копируется, ежедневная проверка свежести копий, внеочередная копия, полное восстановление, восстановление на момент времени, ежемесячная и ежеквартальная проверка восстановления с протоколом. Справочник — «Руководство по развёртыванию», разделы «Резервное копирование», «Восстановление», «Ежемесячная проверка восстановления», «Реплика и архив WAL» и «Восстановление на момент времени»; решения — ADR-014.

Все команды выполняются на узле стенда из каталога, где лежат `docker-compose.yml` и `.env`. Время в именах копий, журналах контейнеров и базе данных — UTC.

2 Когда применять

СИТУАЦИЯ	РАЗДЕЛ
Каждый рабочий день, утром	«Проверка свежести копий»
Перед обновлением, массовым импортом, ручным изменением данных	«Внеочередная копия»
База потеряна или испорчена, восстановление на новом узле	«Полное восстановление»
Ошибочное массовое изменение, нужно состояние на момент до него (стенд с репликой)	«Восстановление на момент времени»
Раз в месяц	«Ежемесячная проверка восстановления»
Раз в квартал (стенд с репликой)	«Ежеквартальная проверка восстановления на момент времени»
Основной сервер PostgreSQL недоступен, реплика исправна	Не этот регламент — `failover.md`

3 Что копируется

МЕХАНИЗМ	ЧТО	КАК ЧАСТО	ГДЕ	СКОЛЬКО ХРАНИТСЯ	ПОТЕРЯ ДАННЫХ ПРИ ВОССТАНОВЛЕНИИ
Сервис `backup` — `pg_dump -Fc`	Вся база `vacation`: сотрудники, заявки, журналы изменений и входов, сеансы, очередь писем	Раз в `BACKUP_INTERVAL` секунд (86400 — сутки) и при каждом старте контейнера	Том `db-backups`, файлы `/backups/vacation-ГГГГММДД-ЧЧММСС.dump`; внеочередные — `vacation-manual-…dump`	`BACKUP_KEEP_DAYS` дней (14), включая внеочередные	Всё после времени копии — до суток
Сервис `basebackup` — `pg_basebackup` (стенд с `docker-compose.replica.yml`)	Физическая копия всего кластера PostgreSQL	Раз в `BASEBACKUP_INTERVAL` секунд (86400) и при каждом старте контейнера	Том `wal-archive`, каталоги `/wal-archive/base/ГГГГММДД-ЧЧММСС/base.tar.gz`	`BASEBACKUP_KEEP` последних копий (7)	Вместе с архивом WAL — до 5 минут
Архив WAL — `archive_command` основного сервера	Сегменты журнала транзакций; `*.history` — линии времени после переключения	Заполненный сегмент или раз в `archive_timeout` (300 секунд), если были изменения	Том `wal-archive`, файлы `/wal-archive/<сегмент>`	До старта самой старой хранимой физической копии; `*.history` — всегда	—
Реплика `postgres-replica`	Горячий резерв: копия базы с отставанием в секунды	Непрерывно	Том `db-replica`	—	Секунды — но реплика повторяет и ошибочные изменения: это не резервная копия

Тома лежат на диске узла (`docker volume inspect -f '{{.Mountpoint}}' <проект>_db-backups`). Копии нужно регулярно переносить на хранилище вне сервера, а том `wal-archive` — размещать на отдельном диске или хранилище: авария диска иначе уничтожит и базу, и копии.

Не копируется и хранится отдельно: `.env` (секреты `JWT_SECRET`, `REPLICATION_PASSWORD`, `SMTP_PASSWORD`, `OIDC_CLIENT_SECRET`, `METRICS_TOKEN`) — в хранилище секретов организации; `docker-compose.override.yml` и изменённые файлы `deploy/`; журналы контейнеров; данные Prometheus (том `prometheus-data`, 30 дней).

Копии содержат персональные данные сотрудников и журнал входов с адресами — храните их с теми же ограничениями доступа, что и базу.

4 Перед началом

ЧТО НУЖНО	ПОДРОБНОСТИ
Доступ	SSH к узлу стенда, право запускать `docker`; для переноса копий — доступ к хранилищу вне сервера
Кто	Проверки и внеочередную копию выполняет администратор стенда. Восстановление с потерей данных согласует ответственный за систему (и служба ИБ, если это часть инцидента — `incident.md`)
Время	Проверка свежести — 5 минут; внеочередная копия — минуты; полное восстановление — простой на время `pg_restore` (длительность — из последнего протокола ежемесячной проверки) плюс 15 минут; восстановление на момент времени — 30–60 минут

Задайте переменную `C` один раз в сеансе:

```
C="docker compose"                                                  # стенд без реплики
C="docker compose -f docker-compose.yml -f docker-compose.replica.yml"     # стенд с репликой и архивом WAL
```

После переключения на реплику (`failover.md`) основным сервером работает `postgres-replica`: в командах `$C exec postgres …` этого регламента подставляйте `postgres-replica`.

5 Проверка свежести копий

5.1 1. Логические копии

```
$C exec backup ls -lt /backups | head -5
$C logs --since 26h backup | grep 'backup:'
```

Ожидаемый результат: самая новая копия `vacation-ГГГГММДД-ЧЧММСС.dump` моложе `BACKUP_INTERVAL` (суток) по UTC; размер не меньше предыдущих (резкое уменьшение — повод проверить копию восстановлением); в журнале строка `backup: /backups/vacation-….dump` и нет `backup: ОШИБКА`.

5.2 2. Копия вне сервера

Проверьте на хранилище организации, что туда перенесена копия не старше суток. Перенос с узла:

```
$C cp backup:/backups/<файл>.dump ./
```

5.3 3. Физические копии, архив WAL и реплика

Только на стенде с репликой:

```
$C exec basebackup ls /wal-archive/base
$C logs --since 26h basebackup | grep 'basebackup:'
$C exec postgres psql -U vacation -Atc "select archived_count, last_archived_wal, now() - last_archived_time,
last_failed_time > last_archived_time from pg_stat_archiver"
$C exec postgres psql -U vacation -Atc "select application_name, state from pg_stat_replication"
```

Ожидаемый результат: новейший каталог `ГГГГММДД-ЧЧММСС` моложе суток, в журнале `basebackup: <метка>` без `basebackup: ОШИБКА`; последний сегмент заархивирован в рабочее время не раньше 5–10 минут назад, последний столбец — `f` или пусто (последняя попытка успешна); `replica1|streaming`.

Не та картина — `alerts.md`: `VacationWalArchiveFailing`, `VacationReplicaDisconnected`; для `backup: ОШИБКА` — «Устранение неполадок» руководства по развёртыванию.

6 Внеочередная копия

6.1 1. Снимите копию

```
$C exec backup sh -c 'pg_dump -Fc -f /backups/vacation-manual-$(date +%Y%m%d-%H%M).dump'
$C exec backup ls -lh /backups | tail -3
```

Ожидаемый результат: команда завершилась без сообщений, появился `vacation-manual-ГГГГММДД-ЧЧММ.dump` размером не меньше последней ежедневной копии.

6.2 2. Проверьте, что копия читается

```
$C exec backup sh -c 'pg_restore -l /backups/<файл>.dump | head -5'
```

Ожидаемый результат: заголовок оглавления архива («Archive created at …», «dbname: vacation»). Ошибка чтения — копия испорчена, повторите шаг 1.

6.3 3. Вынесите копию с сервера

```
$C cp backup:/backups/<файл>.dump ./
```

Перенесите файл на хранилище вне сервера средствами организации. В томе внеочередная копия удаляется через `BACKUP_KEEP_DAYS` дней вместе с ежедневными.

7 Полное восстановление

Заменяет всю базу содержимым копии `pg_dump`. Все изменения после времени копии теряются.

7.1 1. Подготовьте копию и список потерянных изменений

```
$C cp backup:/backups/<файл>.dump ./     # если копия в томе; иначе — с хранилища вне сервера
```

Если текущая база ещё читается, сохраните её внеочередной копией (раздел выше) и выгрузите журнал изменений после времени выбранной копии — по нему кадровая служба повторит потерянные изменения:

```
$C exec -T postgres psql -U vacation -d vacation -c "\copy (select a.ts, e.login, a.action, a.entity, a.entity_id from
audit_events a left join employees e on e.id = a.actor_id where a.ts > '<время копии, UTC>' order by a.ts) to stdout with
csv header" > changes-after-copy.csv
```

7.2 2. Остановите приложение и копирование

```
$C --profile full stop server web backup
```

Ожидаемый результат: три сервиса — `exited`. `backup` останавливается, чтобы очередная копия не снялась с недовосстановленной базы.

7.3 3. Пересоздайте базу и восстановите копию

```
$C exec postgres dropdb -U vacation vacation
$C exec postgres createdb -U vacation vacation
$C cp ./<файл>.dump postgres:/tmp/restore.dump
$C exec postgres pg_restore -U vacation -d vacation /tmp/restore.dump
$C exec postgres rm /tmp/restore.dump
```

Ожидаемый результат: `pg_restore` завершается без вывода. Строка «errors ignored on restore» — восстановление неполное, разберите ошибки выше неё до запуска приложения.

Если `dropdb` отвечает «database “vacation” is being accessed by other users», к базе подключён кто-то ещё (консоль администратора, внешняя программа). Посмотрите, кто, и закройте эти соединения:

```
$C exec postgres psql -U vacation -d postgres -c "select pid, application_name, client_addr from pg_stat_activity where
datname = 'vacation'"
$C exec postgres psql -U vacation -d postgres -c "select pg_terminate_backend(pid) from pg_stat_activity where datname =
'vacation'"
```

7.4 4. Запустите приложение

```
$C --profile full start server web backup
```

Ожидаемый результат: `server` при старте применяет миграции, которых нет в копии (копия более старой версии), и пишет «API планировщика отпусков слушает порт 8080»; `backup` сразу снимает новую копию восстановленной базы.

7.5 5. Проверьте

```
$C exec server wget -qO- http://localhost:8080/readyz     # {"ok":true,"db":"up"}
$C exec postgres psql -U vacation -d vacation -c "select (select count(*) from employees) as employees, (select count(*)
from vacation_requests) as requests, (select max(ts) from audit_events) as last_change_utc"
```

Ожидаемый результат: числа соответствуют копии, последнее изменение журнала — незадолго до времени копии из имени файла. Вход и «График отпусков» в интерфейсе работают.

Что вернулось вместе с копией и требует внимания:

- сеансы: входы после времени копии недействительны — эти сотрудники войдут заново; сеансы, завершённые после копии, снова действуют — если восстановление связано с инцидентом ИБ, завершите их и смените пароли повторно;
- пароли, блокировки и правила организации — на момент копии;
- письма, поставленные в очередь до копии и отправленные после неё, могут уйти повторно.

7.6 Восстановление на новом узле

1. Установите ту же версию, что сняла копию, по разделу «Установка» руководства по развёртыванию (шаги 1–3, без демо-данных). `.env` возьмите из хранилища секретов.
2. Выполните шаги 2–5 этого раздела.
3. Реплику и архив WAL на новом узле поднимайте после восстановления (раздел «Реплика и архив WAL» руководства) — реплика скопирует уже восстановленную базу.

8 Восстановление на момент времени

Только на стенде с `docker-compose.replica.yml`. Физическая копия и архив WAL разворачиваются в отдельный каталог и проигрываются до момента T; рабочая база при этом не меняется, пока вы сами не перенесёте результат.

8.1 1. Определите момент T

Найдите первое ошибочное изменение в журнале изменений («Журнал аудита → Изменения» или запрос ниже) и возьмите T немного раньше него. Столбец `ts` журнала изменений хранится в UTC.

```
$C exec postgres psql -U vacation -d vacation -c "select a.ts, e.login, a.action, a.entity, a.entity_id from audit_events a
left join employees e on e.id = a.actor_id where a.ts > now() - interval '1 day' order by a.ts desc limit 50"
```

Если план — вернуть к T всю базу (вариант А шага 6), остановите `server` сразу: изменения пользователей после T всё равно будут потеряны.

```
$C --profile full stop server
```

8.2 2. Найдите том архива и физическую копию

```
docker volume ls | grep wal-archive          # имя тома: <проект compose>_wal-archive
$C exec basebackup ls /wal-archive/base
```

Ожидаемый результат: каталоги `ГГГГММДД-ЧЧММСС` (время начала копии, UTC) и метки `.started`. Выберите самую позднюю копию, начатую раньше T (T переведите в UTC: 14:55 МСК — 11:55 UTC). Копия, начатая позже T, до T не восстановится. Ниже имя тома — `vacation-planner_wal-archive`; подставьте своё.

8.3 3. Разверните копию в пустой каталог

```
mkdir -p /srv/restore                        # каталог должен быть пустым
docker run --rm -e BASE=<каталог копии> -v vacation-planner_wal-archive:/wal-archive:ro -v /srv/restore:/restore
postgres:17-alpine sh -c '
  tar xzf /wal-archive/base/$BASE/base.tar.gz -C /restore
  touch /restore/recovery.signal
  echo "restore_command = '"'"'cp /wal-archive/%f %p'"'"'" >> /restore/postgresql.auto.conf
  echo "recovery_target_time = '"'"'2026-11-20 14:55:00+03'"'"'" >> /restore/postgresql.auto.conf
  chown -R 70:70 /restore && chmod 700 /restore'
```

В `recovery_target_time` укажите T с часовым поясом. Без этой строки восстановление идёт до последнего заархивированного сегмента.

8.4 4. Запустите восстановление

```
docker run -d --name vp-restore -v /srv/restore:/var/lib/postgresql/data \
  -v vacation-planner_wal-archive:/wal-archive:ro postgres:17-alpine
docker logs -f vp-restore        # «recovery stopping before commit …», «pausing at the end of recovery»
```

Ожидаемый результат: в журнале — восстановление сегментов из архива, затем «recovery stopping before commit ...» и «pausing at the end of recovery». «recovery ended before configured recovery target was reached» — T позже последнего заархивированного сегмента: повторите с меньшим T или без него.

8.5 5. Сверьте данные и завершите восстановление

```
docker exec vp-restore psql -U vacation -d vacation -c "select count(*) from vacation_requests"
docker exec vp-restore psql -U vacation -d vacation -c "select max(ts) from audit_events"
docker exec vp-restore psql -U vacation -c "select pg_wal_replay_resume()"   # данные верны — завершить восстановление
```

Ожидаемый результат: ошибочных изменений нет, последнее изменение журнала — до T (в UTC). Если данные не те — `docker rm -f vp-restore`, очистите `/srv/restore` и повторите с шага 3 с другим T. После `pg_wal_replay_resume()` экземпляр `vp-restore` становится обычным сервером (архивирование в нём выключено — в общий архив он не пишет).

8.6 6. Перенесите результат

ВАРИАНТ	КОГДА	КАК
А. Вся база — на момент T	Ошибка затронула много данных; изменения после T можно повторить вручную	Выгрузите базу из `vp-restore` и выполните «Полное восстановление» с этим файлом. Перед этим выгрузите журнал изменений после T (шаг 1 «Полного восстановления»)
Б. Точечно	Испорчены отдельные записи, остальная работа после T должна сохраниться	Выгрузите нужные строки из `vp-restore` (`\copy (select …) to stdout with csv header`) и исправьте данные через интерфейс (кадровая служба) — исправления попадут в журнал изменений. Правку SQL в рабочей базе выполняет только разработчик по согласованию с ответственным за систему

Выгрузка для варианта А:

```
docker exec vp-restore pg_dump -U vacation -d vacation -Fc -f /tmp/pitr.dump
docker cp vp-restore:/tmp/pitr.dump ./pitr—<дата>.dump
```

8.7 7. Уберите временный экземпляр

```
docker rm -f vp-restore && rm -rf /srv/restore
$C --profile full start server              # если останавливали на шаге 1 и не выполняли вариант А
```

9 Ежемесячная проверка восстановления

Последняя копия восстанавливается в отдельную базу `restore_check` на рабочем сервере и сверяется с рабочей базой. Выполняйте вне часов пик: восстановление нагружает сервер, а на стенде с репликой ещё и реплику и архив WAL.

9.1 1. Восстановите последнюю копию

```
$C exec backup sh -c 'ls -lt /backups/*.dump | head -1'                        # какая копия проверяется
$C exec postgres createdb -U vacation restore_check
time $C exec backup sh -c 'pg_restore -d restore_check $(ls -t /backups/*.dump | head -1)'
```

Ожидаемый результат: `pg_restore` без вывода, `time` показывает длительность — это оценка простоя при полном восстановлении.

9.2 2. Сверьте с рабочей базой

```
Q="select (select count(*) from employees) as employees, (select count(*) from vacation_requests) as requests, (select
max(ts) from audit_events) as last_change_utc"
$C exec postgres psql -U vacation -d restore_check -c "$Q"
$C exec postgres psql -U vacation -d vacation -c "$Q"
```

Ожидаемый результат: в копии сотрудников и заявок не больше, чем в рабочей базе, разница объясняется работой после времени копии; последнее изменение в копии — незадолго до времени копии из имени файла.

9.3 3. Удалите проверочную базу

```
$C exec postgres dropdb -U vacation restore_check
```

9.4 4. Заполните протокол

ПОЛЕ	ЗНАЧЕНИЕ
Дата и время проверки (UTC), исполнитель	
Проверенная копия: файл, размер, возраст на момент проверки	
Длительность `pg_restore`	
Ошибки `pg_restore`	нет / текст
Сотрудников: копия / рабочая база	
Заявок: копия / рабочая база	
Последнее изменение в копии (UTC)	
Копия не старше суток есть на хранилище вне сервера	да / нет, где
Итог	пройдено / не пройдено
Замечания и меры	

Проверка пройдена, если `pg_restore` завершился без ошибок, числа и время последнего изменения согласуются с временем копии и копия есть вне сервера. Не пройдена — внеочередная копия, повтор проверки на ней и разбор причины (`backup: ОШИБКА`, место на томе, доступ к БД).

10 Ежеквартальная проверка восстановления на момент времени

Выполняется на стенде с репликой по разделу «Восстановление на момент времени» без остановки `server` (шаг 1 — только выбор T) и без переноса результата (шаг 6 пропускается).

1. Возьмите T около часа назад в рабочее время, запишите его в UTC.
2. Выполните шаги 2–5. Вместо поиска ошибочных изменений сверьте журнал изменений — он только дополняется:

```
$C exec postgres psql -U vacation -d vacation -Atc "select count(*) from audit_events where ts ⩽ '<T в UTC, без пояса>'"
docker exec vp-restore psql -U vacation -d vacation -Atc "select count(*), max(ts) from audit_events"
```

Ожидаемый результат: числа совпадают или отличаются на единицы (транзакции, начатые до T и завершённые после), `max(ts)` — не позже T.

3. Выполните шаг 7 (без запуска `server` — он не останавливался).
4. Заполните протокол: дата и исполнитель, T, использованная физическая копия, время от запуска `vp-restore` до «pausing at the end of recovery», числа журнала изменений, итог и замечания.

11 Проверка

ОПЕРАЦИЯ	ГОТОВО, КОГДА
Проверка свежести	Логическая копия моложе суток, есть копия вне сервера; на стенде с репликой — физическая копия моложе суток, архив без ошибок, `replica1|streaming`
Внеочередная копия	`pg_restore -l` читает оглавление, файл вынесен с сервера
Полное восстановление	`/readyz` — `{"ok":true,"db":"up"}`, числа и время последнего изменения соответствуют копии, вход и график работают, `backup` снял новую копию
Восстановление на момент времени	Ошибочных изменений нет, последнее изменение журнала — до T; результат перенесён, `vp-restore` и `/srv/restore` удалены
Ежемесячная и ежеквартальная проверка	Протокол заполнен, итог «пройдено»

12 Если не получилось

ПРИЗНАК	ДЕЙСТВИЕ
Свежей копии нет, в журнале `backup: ОШИБКА`	Место на томе `db-backups` (`df -h`), доступ к БД (`$C logs postgres`); внеочередная копия вручную; дальше — «Устранение неполадок» руководства
`pg_restore` полной копии завершился с ошибками	Не запускайте приложение на неполной базе: повторите шаг 3 с предыдущей копией; при повторе — эскалация разработчикам с текстом ошибок
После восстановления `server` не стартует	`$C logs --no-log-prefix server` : ошибка миграции — копия от более новой версии, чем образы стенда; восстановите на той версии, что сняла копию (`update.md` , «Откат»)
Восстановление на момент времени не доходит до T	В архиве нет нужных сегментов: `VacationWalArchiveFailing` в тот период или копия начата позже T — возьмите более раннюю физическую копию или полное восстановление из `pg_dump`
Проверка восстановления не пройдена второй раз подряд	Эскалация ответственному за систему: копирование не обеспечивает восстановление

13 Запись в журнал эксплуатации

ПОЛЕ	ПРИМЕР
Дата, время (UTC) и исполнитель	2026-11-20 05:30, администратор стенда
Операция	Проверка свежести / внеочередная копия / полное восстановление / восстановление на момент T / ежемесячная или ежеквартальная проверка
Копия	Имя файла или каталога физической копии, размер, место хранения вне сервера
Результат	Числа сверки, длительность, итог; для восстановления — время простоя и потерянный период
Решение и согласование	Кто согласовал восстановление с потерей данных; ссылка на протокол или отчёт об инциденте